Assignment 3: Graph Algorithms

Dauletkyzy Bayan | SE-2523
Design and Analysis of Algorithms | 22 October 2026

1. Implementation and complexity

One adjacency-list Graph supports directed and undirected graphs, integer weights, loops, parallel edges and isolated vertices. Undirected edges are stored once in the edge list and twice in adjacency lists. Kruskal and Prim return a forest when the graph is disconnected. DFS uses explicit stacks instead of recursion.

Let n = |V| and m = |E|. Extra space below excludes the input graph but includes algorithm data and returned results. DSU uses union by size and path compression. The bounds use log(m + 1) so that empty graphs are covered.

Operation	Time	Extra space	Reason
DSU makeSet	O(n)	O(n)	Initializes parent and size arrays.
DSU find / union	O(alpha(n)) amortized	O(1) per call; O(n) stored	Size balancing and compression shorten parent chains; find is iterative.
Kruskal	O(n + m log(m+1) + m alpha(n))	O(n + m)	Copies and sorts edges, then checks endpoints with DSU.
Prim (lazy edge queue)	O(n + m log(m+1))	O(n + m)	Scans adjacency lists; at most O(m) queue entries are inserted and removed.
Kosaraju	O(n + m)	O(n + m)	Two DFS passes; the reversed graph needs O(n + m) storage.
Kahn	O(n + m)	O(n)	Indegrees, queue and order; visits every vertex and outgoing edge.
DAG shortest paths	O(n + m)	O(n)	Topological sorting and one relaxation pass; distance and parent arrays.
Tarjan	O(n + m)	O(n)	One DFS, arrays and two explicit stacks; no reversed graph.
Condensation + longest path	O(n + m)	O(n + m)	Includes SCC discovery, condensed graph construction and DAG relaxation.

2. Which algorithm when?

Situation	Choice and reason
Sparse undirected graph; edge list	Kruskal: sorting and DSU fit the input representation directly.
Undirected graph; adjacency lists	Prim: grows a tree using neighbors and a priority queue. Restart for disconnected components.
Dense undirected graph	Compare measured queue and sorting work. This lazy Prim has no automatic asymptotic advantage over Kruskal. A separate matrix-based Prim can run in O(n squared), but is not implemented here.
Directed graph; cycles possible	Kosaraju or Tarjan for SCCs. Tarjan scans fewer edges; Kosaraju has a simpler two-pass structure.
Dependencies expected to form a DAG	Kahn produces a valid order or reports a cycle.
Weighted DAG, including negative weights	Topological relaxation for shortest paths; flipped weights for source-based longest paths.

Correctness and bonus tasks

3. Why the MST edges are safe

A cut divides the vertices into two sets. Consider a cut that does not cross any edge already selected, and choose a lightest edge e across it. If a minimum spanning tree containing the selected edges does not contain e, adding e creates a cycle. That cycle contains another edge f across the same cut, and f cannot be lighter than e. Replacing f with e preserves a spanning tree and does not increase its weight, so e is safe. Kruskal applies this idea to distinct DSU components while processing edges in increasing weight order. Prim applies it to the cut between visited and unvisited vertices, and the same argument works separately inside each connected component to produce a minimum spanning forest.

4. Why one DAG relaxation pass is enough

In a topological order, every edge goes from an earlier vertex to a later vertex. Each vertex with indegree zero starts at distance zero, representing a virtual super-source. These source distances form the base case of the induction. Assume that every vertex before v already has its correct shortest distance. Every path into v ends with an edge from one of those earlier vertices, and that edge has already been relaxed. Therefore v has its correct distance when it is processed, and relaxing its outgoing edges extends the induction. Negative weights are safe because a DAG has no cycles, while a general graph requires a different algorithm and a reachable negative cycle can make a finite shortest distance undefined. The parent array records each improving predecessor so that a shortest path can be restored.

5. Bonus A: Tarjan SCC

Tarjan assigns each vertex a discovery index. Its lowlink value records the earliest index reachable through DFS descendants and relevant links to active vertices. One stack simulates DFS calls; another holds vertices whose SCC is not yet complete. An edge to a vertex already removed from the active stack does not reduce lowlink. When lowlink[v] equals index[v], active vertices are popped through v and assigned to one component.

The benchmark compares the component partitions using mappings in both directions, because different numeric component IDs are valid. All ten benchmark configurations agree with Kosaraju. Tarjan scans m edges, whereas Kosaraju scans 2m in its two DFS passes: a 50% saving in this counter. Kosaraju also constructs a reversed graph; that construction is outside the DFS edge_scans count. Both algorithms remain O(n + m).

6. Bonus B: condensation and critical path

CriticalPath.condense replaces every SCC with one vertex, removes edges within an SCC and preserves the weights of edges between SCCs. Parallel inter-component edges are retained. The condensation is acyclic: a cycle between different components would make them mutually reachable, so they should have been a single SCC.

CriticalPath.longestPath processes this DAG in topological order and minimizes sums of negated weights using long arithmetic. Negating the resulting distance gives the longest distance. The convention is a path from any indegree-zero source to any endpoint, allowing a zero-length path. Internal SCC weights are not included, so this is a path in the condensation, not a solution to the original graph's longest simple path problem. The method returns the length.

For components {0,1}, {2,3} and {4}, the test has an inter-component route of weight 4 + 6 = 10 and a direct route of weight 3; the expected result is 10. In scheduling, nonnegative durations and precedence constraints give a critical path that determines the earliest possible project completion. A DAG allows one pass because prerequisites come first. Finding a longest simple path in a general graph is NP-hard; allowing repeated vertices can instead make the length unbounded when a positive cycle is reachable.

Measured work and complexity ratios

7. Experimental setup

Five sizes (1,000; 3,162; 10,000; 31,623; 100,000) use m = 4n and m = 20n. Each configuration has an undirected connected graph, a directed graph and a DAG. Each generation starts with new Random(42). When both endpoints coincide, only the second endpoint is sampled again. Weights are 1 to 1,000. Each algorithm is measured five times; the CSV stores the median time. The main CSV has 50 rows and the Tarjan CSV has 10 rows.

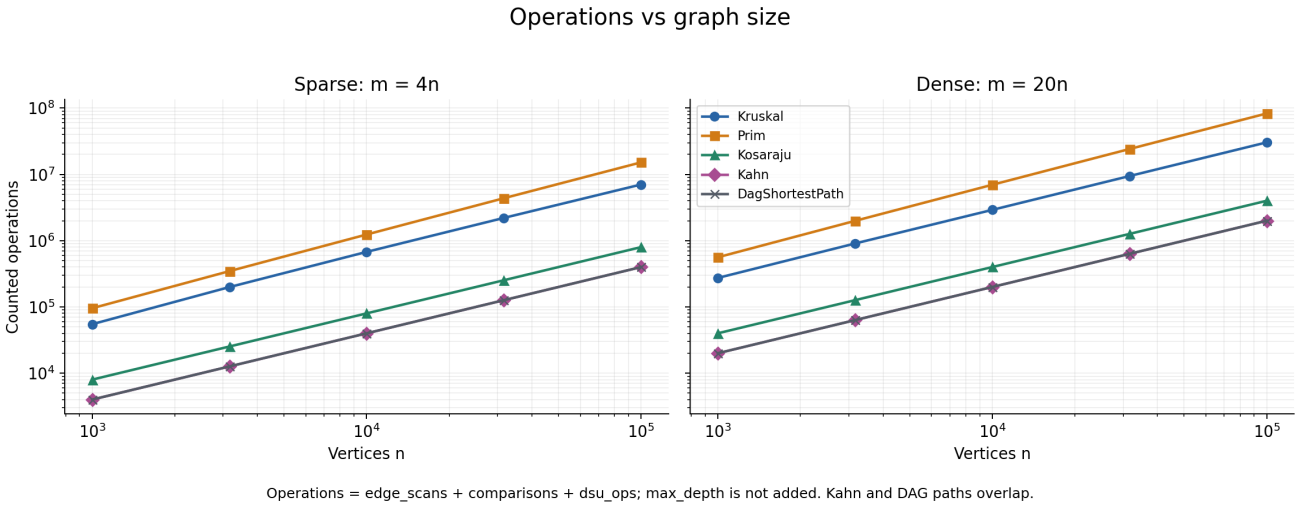


Figure 1. Counted operations are edge_scans + comparisons + dsu_ops. max_depth is a maximum, not a count to add. Equal weighting is a plotting convention; these counters are not CPU instructions. Kahn and DAG relaxation have identical counted work. Their preprocessing and other work are not fully represented by this sum.

8. Counters divided by complexity formulas

Values in each sequence correspond to the five sizes in increasing order. c = comparisons, s = edge_scans, d = dsu_ops. Theoretical bounds concern total work; a selected counter measures only part of it.

Algorithm / density	Ratio	Five measured values
Kruskal / 4n	c / (m log2 m)	0.8914, 0.9069, 0.8692, 0.8091, 0.7447
Kruskal / 20n	c / (m log2 m)	0.9027, 0.8464, 0.7839, 0.7200, 0.6662
Kruskal / 4n	d / m	2.2902, 2.5189, 2.6818, 2.7317, 2.7483
Kruskal / 20n	d / m	0.5800, 0.6062, 0.5647, 0.7540, 0.8936
Prim / 4n	c / (m log2 m)	1.8291, 1.8654, 1.8952, 1.9210, 1.9344
Prim / 20n	c / (m log2 m)	1.8248, 1.8468, 1.8657, 1.8779, 1.8896
Kosaraju / 4n; 20n	s / (n + m)	1.6000 at every size; 1.9048 at every size
Kahn / 4n; 20n	s / (n + m)	0.8000 at every size; 0.9524 at every size
DAG paths / 4n; 20n	s / (n + m)	0.8000 at every size; 0.9524 at every size
Tarjan / 4n; 20n	s / (n + m)	0.8000 at every size; 0.9524 at every size

Traversal ratios are exactly constant. Prim comparison ratios are stable within a small range. Kruskal ratios vary because it stops scanning after the tree is complete; sorting still processes every edge, and repeated bounded weights affect comparison behavior. These measurements support the stated bounds on the tested inputs but do not prove them. DSU counts calls rather than parent traversals, so d/m cannot establish the inverse-Ackermann bound.

Edge scans and fitted slopes

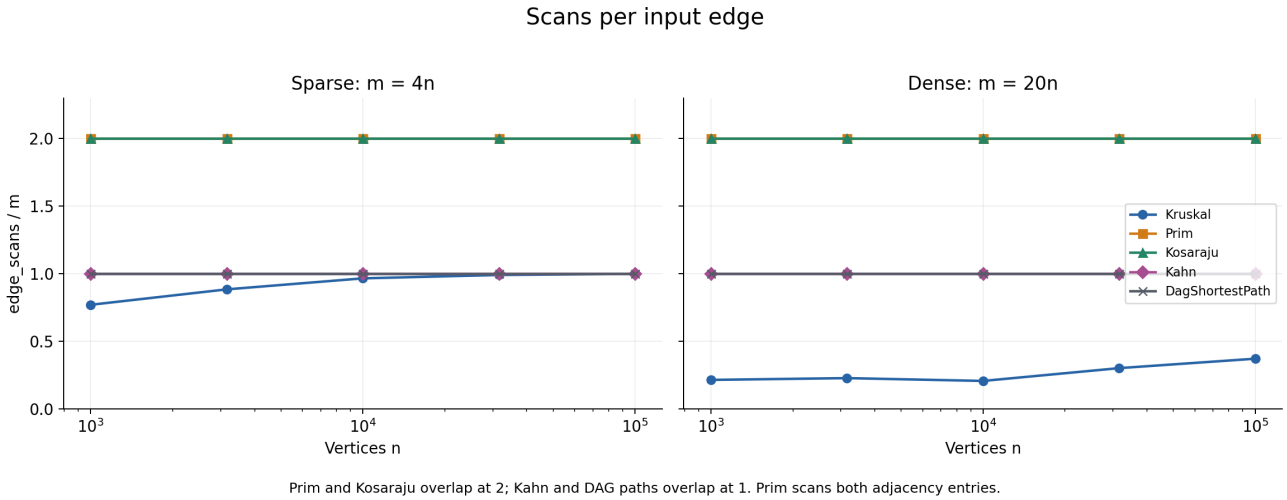


Figure 2. Prim scans both stored directions of every undirected edge, giving 2m. Kosaraju scans every directed edge twice. Kahn and DAG relaxation scan m. The flat ratios confirm linear growth of these scans at fixed density.

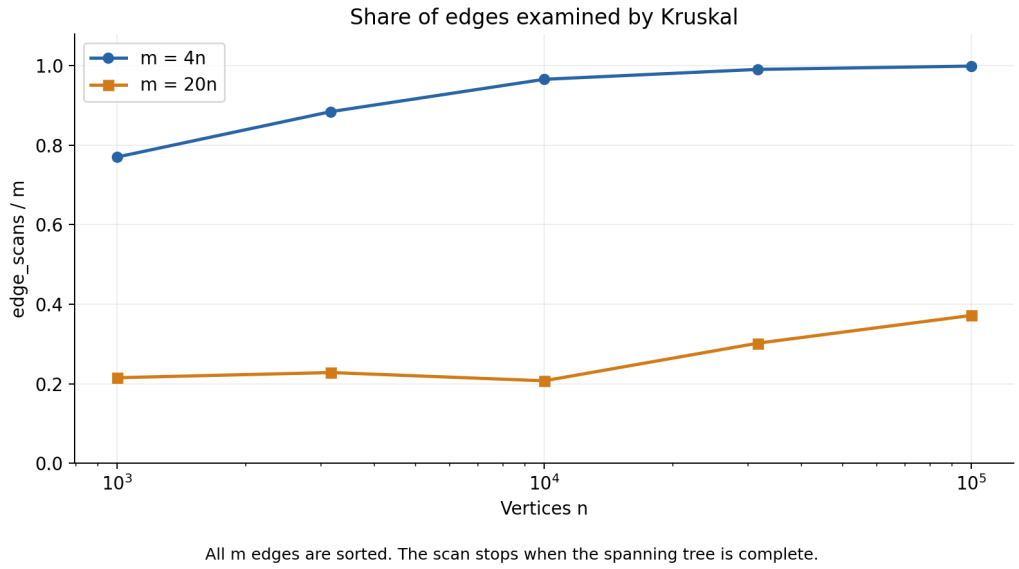


Figure 3. Kruskal scans 77.05% to 99.917% of sparse edges, but only 20.736% to 37.179% of dense edges in this experiment. Its early stop saves scan and DSU work on dense graphs. Sorting all edges is still necessary; the share alone is not a complete speed comparison with Prim.

9. Slope check

Counter	m = 4n slope	m = 20n slope
Kruskal edge_scans	1.0550	1.1194
Prim / Kosaraju / Kahn / DAG / Tarjan edge_scans	1.0000 each	1.0000 each
Kruskal comparisons	1.0545	1.0159
Prim comparisons	1.1079	1.0903

Slopes are least-squares fits of log10(counter) against log10(n). A scan slope of 1.00 means proportional growth at fixed density. The assignment links Kruskal scan slope to sorting, but sorting is recorded in comparisons, not edge_scans. Its measured scan slope above one reflects changing scan share over this finite range. The comparison slopes are reported separately; repeated weights and finite sizes affect them.

Validation, timing and reproduction

10. Correctness checks

The JUnit suite includes random DSU checks against a simple group-label reference, 100 random connected graphs comparing both MST weights and n - 1 selected edges, SCC checks using BFS mutual reachability on graphs with at most 200 vertices, topological-order validation and cycle rejection, and DAG shortest-path comparisons with Bellman-Ford including negative weights. A Kosaraju path test uses 100,000 vertices to exercise the explicit stack. The input files cover a disconnected undirected graph, a directed cycle and a DAG.

Bonus tests compare Tarjan and Kosaraju on random graphs and check condensation and longest-path lengths, including an extreme negative int weight. The benchmark additionally verifies all supplied reference results, both MST totals, topological orders, expected scan counts and equivalence of SCC partitions. These benchmark checks completed successfully for all configurations. A fresh full JUnit run is required when preparing the final release.

All ten reference configurations match. For n = 100,000 and m = 2,000,000, the MST total is 3,056,681, the SCC count is 1 and the sum of DAG distances is 23,512,087. Complete results are stored in the two CSV files.

11. Wall-clock measurements

Algorithm	100,000 / 400,000 (ms)	100,000 / 2,000,000 (ms)
Kruskal	231.178	904.623
Prim	344.007	2551.825
Kosaraju	151.130	749.980
Tarjan	70.805	230.981
Kahn	20.126	79.361
DAG shortest paths	37.519	254.991

Measurements were taken on Windows with Microsoft OpenJDK 17.0.18. On the largest dense graph, Kruskal took 904.623 ms and Prim took 2551.825 ms; this describes this implementation and run, not every graph or computer. Small-input times are noisy and can decrease as n grows because of JVM warm-up and background activity. Five-run medians reduce sensitivity to individual slow runs, but there is no separate warm-up phase. Hardware, JVM behavior and garbage collection affect timings, so operation counts are the main evidence. Kahn timing includes order validation, and DAG timing includes topological preprocessing and distance summation; its edge_scans counts relaxation only. SCC partition comparison is outside timed runs.

12. Reproduce and acknowledge assistance

From the project root, run `mvn clean test`, then `java -cp target/classes benchmark.BenchmarkRunner`. The benchmark regenerates both CSV files. With Python and Matplotlib installed, `python plot_results.py` regenerates the three plots and analysis. README.md contains setup and output details.

AI assistance was used to explain requirements and algorithms, draft and debug code and tests, diagnose the random-generator mismatch, and prepare plots and report text. Benchmark values came from the project run on the student computer. The student is responsible for reviewing the implementation and explaining it at the defense.

Sources: course specification DAA_Assignment3_Graphs(2).pdf; project implementation discussed during development; student-generated results/results.csv and results/tarjan.csv. No benchmark measurements were invented.